

build more accurate and personalized recommendations for both **movies** and **grocery products**.

Basic Feature Engineering Techniques:

1. Average Rating / Average Purchase Frequency

- **What it is:** Measures how generally liked or purchased a movie or product is.
- **Why it's useful:** Higher averages indicate more popular items.
- **Formula:**
 - **Average Rating (Movies):**

$$\text{average_rating} = \frac{\sum(\text{ratings})}{\text{count}(\text{ratings})}$$

- **Average Purchase Frequency (Groceries):**

$$\text{average_purchase_frequency} = \frac{\sum(\text{purchases})}{\text{count}(\text{purchases})}$$

- **Sample Code:**

```
python

df['average_rating'] = df.groupby('movie_id')['rating'].transform('mean')
df['average_purchase_frequency'] = df.groupby('product_id')
['purchase_count'].transform('mean')
```

2. Rating / Purchase Standard Deviation (Dispersion)

- **What it is:** Measures how much the ratings or purchase counts vary.
- **Why it's useful:** A higher standard deviation means more disagreement in ratings, indicating a polarizing item.
- **Formula:**
 - **Standard Deviation of Rating (Movies):**

$$\text{std_rating} = \sqrt{\frac{\sum(\text{rating} - \text{mean_rating})^2}{\text{count}(\text{ratings})}}$$

- **Standard Deviation of Purchase Frequency (Groceries):**

$$\text{std_purchase_frequency} = \sqrt{\frac{\sum(\text{purchase_frequency} - \text{mean_purchase_frequency})^2}{\text{count}(\text{purchases})}}$$

- **Sample Code:**

```
python

df['std_rating'] = df.groupby('movie_id')['rating'].transform('std')
df['std_purchase_frequency'] = df.groupby('product_id')
```

```
['purchase_count'].transform('std')
```

3. Skewness of Ratings / Purchases

- **What it is:** Measures whether ratings or purchases are mostly on the high or low end.
- **Why it's useful:** Positive skew means higher ratings or purchases are more common, and negative skew means the opposite.
- **Formula:**

$$\text{skew} = \frac{(\text{mean} - \text{median})}{\text{std_deviation}}$$

- **Sample Code:**

```
python

from scipy.stats import skew
df['skew_rating'] = df.groupby('movie_id')['rating'].transform(lambda x: skew(x))
df['skew_purchase_frequency'] = df.groupby('product_id')
['purchase_count'].transform(lambda x: skew(x))
```

4. Recency

- **What it is:** Measures how recent a rating or purchase is.
- **Why it's useful:** More recent interactions are often more relevant for making recommendations.
- **Formula:**

$$\text{recency} = \text{current_time} - \text{time_of_rating_or_purchase}$$

- **Sample Code:**

```
python

current_time = pd.to_datetime('2025-06-30') # today's date
df['recency'] = (current_time - df['timestamp']).dt.days
```

5. Purchase Frequency

- **What it is:** Measures how frequently a product is bought.
- **Why it's useful:** Frequently bought items are more likely to be recommended.
- **Formula:**

$$\text{purchase_frequency} = \frac{\text{count}(\text{purchases})}{\text{total_time_period}}$$

- **Sample Code:**

```
python
```

```
df['purchase_frequency'] = df.groupby('product_id')['purchase_count'].transform('sum')
```

6. Weighted Rating

- **What it is:** Adjusts the rating based on the number of ratings or purchases to prioritize more reliable data.
- **Why it's useful:** Movies with more ratings or products with more purchases are more reliable recommendations.
- **Formula:**

$$\text{weighted_rating} = \frac{\text{rating} \times \text{number_of_ratings}}{\text{max_ratings}}$$

- **Sample Code:**

```
python
```

```
df['weighted_rating'] = df['rating'] * df['purchase_count'] /  
df['purchase_count'].max()
```

7. User Activity Level

- **What it is:** Measures how active a user is based on their interactions (ratings or purchases).
- **Why it's useful:** Active users provide more reliable data for making recommendations.
- **Formula:**

$$\text{user_activity_level} = \frac{\text{count}(\text{ratings})}{\text{total_users}}$$

- **Sample Code:**

```
python
```

```
df['user_activity_level'] = df.groupby('user_id')['rating'].transform('count')
```

8. Category-Level Features

- **What it is:** Features that reflect the popularity or behavior within a category (e.g., movie genre, product category).
- **Why it's useful:** Helps to recommend more items from the user's favorite categories.
- **Formula:**

$$\text{category_purchase_count} = \sum(\text{purchases_by_category})$$

- **Sample Code:**

```
python

df['category_purchase_count'] = df.groupby('category')
['purchase_count'].transform('sum')
```

9. Interaction Features (User x Category or User x Movie)

- **What it is:** Captures interactions between the user and the item, like user preferences for certain categories.
- **Why it's useful:** Personalized recommendations based on user-category interactions.
- **Formula:**

$$\text{user_category_interaction} = \text{user_id} \text{ joined with category}$$

- **Sample Code:**

```
python

df['user_category_interaction'] = df['user_id'].astype(str) + '-' + df['category']
```

10. Clustering Features

- **What it is:** Groups users or items into clusters based on similarities in their behaviors.
- **Why it's useful:** Users with similar tastes or items with similar features can help make better recommendations.
- **Formula:**

$$\text{user_cluster_label} = \text{cluster based on ratings or purchases}$$

- **Sample Code:**

```
python

from sklearn.cluster import KMeans
kmeans = KMeans(n_clusters=3)
df['user_cluster_label'] = kmeans.fit_predict(df[['rating']])
```

Advanced Feature Engineering Techniques:

11. Collaborative Filtering (User-Item Matrix)

- **What it is:** Captures user-item interactions and uses techniques like **matrix factorization** to predict ratings for unseen items.
- **Why it's useful:** Collaborative filtering is widely used for making recommendations based on user preferences.
- **Advanced Techniques:**
 - **SVD** (Singular Value Decomposition) and **ALS** (Alternating Least Squares).

- **Sample Code (SVD using Scikit-surprise):**

```
python

from surprise import SVD, Dataset, Reader
from surprise.model_selection import train_test_split

data = Dataset.load_from_df(df[['user_id', 'movie_id', 'rating']], Reader(rating_scale=(1, 5)))
trainset, testset = train_test_split(data, test_size=0.2)
model = SVD()
model.fit(trainset)
predictions = model.test(testset)
```

12. Content-Based Filtering (Item Features)

- **What it is:** Uses item-specific features (e.g., movie genre, product category) to recommend similar items.
- **Why it's useful:** Effective when users have no prior ratings or interactions.
- **Advanced Techniques:**
 - **TF-IDF** Vectorization or **Word2Vec** to convert item descriptions into feature vectors.
- **Sample Code (TF-IDF for Item Descriptions):**

```
python

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(df['movie_description'])
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

13. Hybrid Recommendation Systems

- **What it is:** Combines collaborative filtering and content-based methods to leverage the strengths of both.
- **Why it's useful:** Combining methods helps when one method alone isn't sufficient.
- **Advanced Techniques:**
 - **Weighted Hybrid** or **Switching Hybrid**.
- **Sample Code (Weighted Hybrid):**

```
python

df['hybrid_score'] = 0.5 * df['collaborative_score'] + 0.5 * df['content_based_score']
```

14. Factorization Machines (FM)

- **What it is:** A machine learning model that generalizes matrix factorization, capturing interactions between users and items.

- **Why it's useful:** Works well for high-dimensional, sparse data.
- **Sample Code (using fastFM):**

```
python

from fastFM import als
model = als.FMRegression(rank=10, l2_reg_w=0.1, l2_reg_V=0.1)
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

15. Deep Learning-Based Recommendation Systems

- **What it is:** Uses neural networks to learn complex patterns from large datasets.
- **Why it's useful:** Deep learning can capture intricate relationships that traditional methods may miss.
- **Sample Code (Neural Network for Recommendations):**

```
python

from keras.models import Model
from keras.layers import Input, Embedding, Flatten, Dot, Dense

user_input = Input(shape=(1,))
item_input = Input(shape=(1,))
user_embedding = Embedding(input_dim=num_users, output_dim=50)(user_input)
item_embedding = Embedding(input_dim=num_items, output_dim=50)(item_input)
dot_product = Dot(axes=1)([user_embedding, item_embedding])
dense_layer = Dense(1)(dot_product)
model = Model(inputs=[user_input, item_input], outputs=dense_layer)
model.compile(optimizer='adam', loss='mean_squared_error')
model.fit([train_user_data, train_item_data], train_ratings, epochs=10)
```

16. Temporal Dynamics

- **What it is:** Accounts for changes in user preferences over time, such as the **decay of older ratings**.
- **Why it's useful:** Captures evolving user tastes and trends.
- **Sample Code (Time Decay):**

```
python

df['time_decay_factor'] = (current_time - df['timestamp']).dt.days
df['decay_adjusted_rating'] = df['rating'] / (1 + df['time_decay_factor'] / 365)
```

17. Context-Aware Recommendations

- **What it is:** Incorporates **external context**, such as **location**, **time of day**, or **device type**, to improve recommendations.
- **Why it's useful:** Context can significantly influence user behavior (e.g., a user might prefer movies on a rainy day).